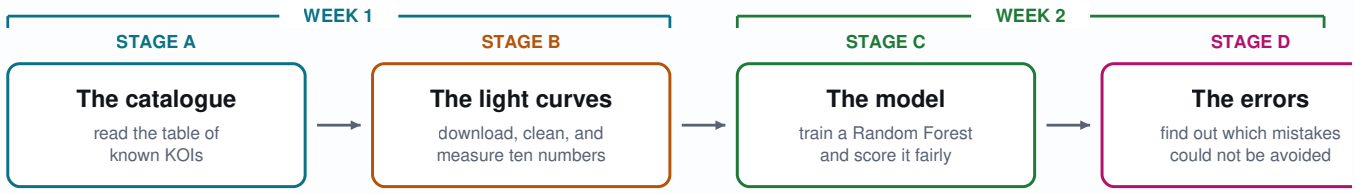
PROJECT BRIEF · 2 WEEKS

Can a computer spot a fake planet?

Machine learning applied to Kepler transit light curves

What fraction of Kepler's false positives can be identified from the shape of the light curve alone?

About half of the signals that look like a planet transit are not planets. Most are **eclipsing binaries**: two stars orbiting each other and passing in front of one another. Sometimes it is the star you are looking at, and sometimes it is a fainter pair of stars nearby whose light falls on the same camera pixels. In this project you will measure ten numbers from each light curve, train a Random Forest to tell planets from false positives using only those ten numbers, and then work out which false positives cannot be found this way and why.



Each stage matches one Part of the notebook.

Stage D is where you find the scientific result. Stages A to C build the tools you need to get there.

WHY THIS QUESTION HAS A REAL ANSWER

A light curve is a record of how much light reached the telescope at each moment. It contains no information about *where* in the field of view that light was lost. A pair of eclipsing stars a little way from your target produces a dip of the same shape and the same depth as a planet, so no measurement made from brightness alone can separate the two. This means your model has a ceiling it cannot pass, and you can measure where that ceiling is.



Two stars eclipsing each other nearby produce the same dip in your light curve. No feature measured from brightness alone can tell them apart.

By the end you will be able to say, using your own numbers, how many false positives can be found from light-curve shape, how many cannot, and what extra data would be needed to find the rest.

WHAT YOU NEED TO KNOW FIRST

- How to download a Kepler light curve with `lightkurve` and plot brightness against time.
- How to remove instrument trends and the star's own brightness changes. This is covered in the detrending notes from earlier in the project.
- How to fold a light curve on a period, and why folding makes a shallow dip easier to see.
- Enough pandas to read a CSV file, select rows, and group them.

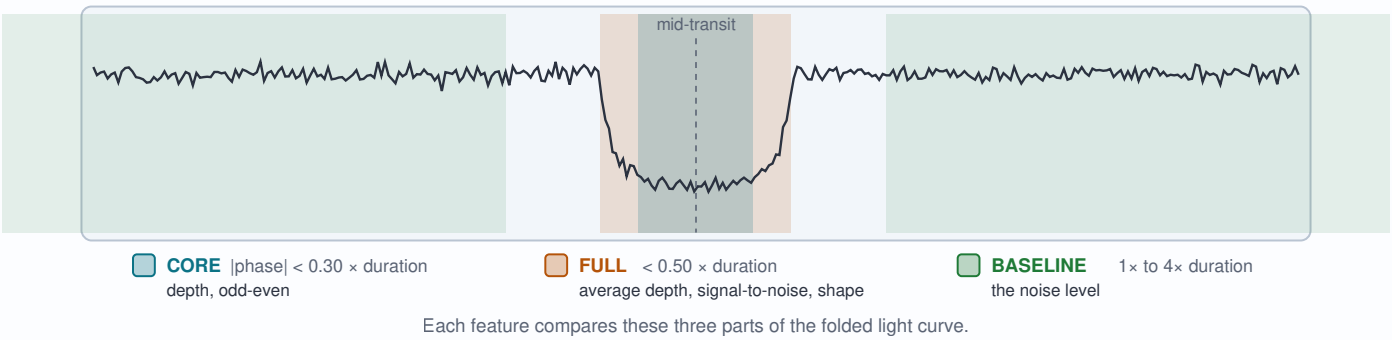
THINGS TO WATCH OUT FOR

- The catalogue contains some columns that give away the answer. Finding them is part of the work.
- Half the objects are false positives, so a high accuracy does not mean much on its own.
- Downloading a hundred targets takes time and uses a lot of data. Start with a small number and increase it once everything works.
- A result you cannot explain in terms of physics is not yet finished.

REFERENCE

The ten features

Every one is measured from the folded light curve. None comes from the catalogue.



Each feature compares these three parts of the folded light curve.

NAME IN THE CODE	HOW IT IS CALCULATED	WHAT IT TELLS YOU
depth_ppm	$\text{median}(F_{base}) - \text{median}(F_{core})$	How much light is blocked. Planets almost never block more than about 3% of the star (30 000 ppm). A much deeper dip usually means one star passing in front of another.
duration_hr	taken from the KOI table	How long the dip lasts, in hours. It sets the size of all three windows and is needed for the density check below.
n_transits	number of separate dips seen	How many times the dip was observed. With only two or three you cannot be confident the signal really repeats. With forty, you can.
scatter_oot_ppm	$\text{std}(F_{base}) \times 10^6$	How noisy the star is outside the transit. A large value means a faint or active star, so every other measurement for that object is less reliable.
snr	$(\delta_{full}/\sigma) \times \sqrt{N_{in}}$	Signal-to-noise of the transit once all the dips are stacked. Most of the objects your model gets wrong will have a low value here.
v_shape	$\delta_{core} / \delta_{full}$	Whether the bottom of the dip is flat or pointed. A planet crossing the middle of the star gives a flat-bottomed U shape (about 1.05); two stars clipping each other give a pointed V (above 1.3).
odd_even_sigma	$\lvert\delta_{odd} - \delta_{even}\rvert / \sigma$	Two stars orbiting each other give a deep eclipse and then a shallower one, so if the listed period is twice the real one, the odd-numbered dips differ from the even-numbered ones. A planet's dips are all the same depth.
secondary_ppm	depth measured at phase 0.5	Whether the star dims again half an orbit later. A planet's night side is much too faint for Kepler to detect, so a real dip here means there is a second star.
depth_scatter	$\text{std}(\delta_n) / \text{mean}(\delta_n)$	Whether the depth stays the same from one dip to the next. A planet blocks the same amount of light every orbit, so a changing depth points to blending, starspots, or detrending that has gone wrong.
rho_star_cgs	$3\pi(a/R_*)^3 / GP^2$, with $a/R_* = P/\pi T$	The density the star would need for this transit to make sense. If a star known to be Sun-like comes out as dense as a red giant, the dip is being diluted by a second star.

Notation. F_{core}, F_{full} and F_{base} are the brightness values that fall inside the three windows shown above. δ is a depth, σ is the scatter of the baseline, N_{in} is the number of points inside the transit, P is the orbital period and T is the transit duration. For the density, work in cgs units with $G = 6.674\times10^{-8}$ and both times in seconds. A star like the Sun should come out at about 1.4 g/cm^3.

WEEK 1

What you do in the first week

Stage A is short. Stage B is the longest part of the whole project.

STAGE A — The catalogue, and the clue hidden in it

Notebook Part 1. You will not open a light curve yet. First you build a classifier from the catalogue numbers only, get a very high score, and then find out why that score was not real.

- Download the KOI table from the NASA Exoplanet Archive. Keep only the rows marked CONFIRMED and FALSE POSITIVE, and drop the ones still marked CANDIDATE.
- Plot the depth, the period and the impact parameter for both groups, so you can see how much they overlap before you model anything.
- Work out what fraction of the objects are false positives. This is the score you would get by guessing, and it is what you have to beat.
- Go through the columns and set aside every column that was filled in *after* somebody decided the answer. There are nine of them.
- Train a Random Forest on the eight remaining catalogue columns, splitting the data by star rather than by row. Write down the recall you get.
- Add `koi_score` back in on purpose and run it again. Look at what happens to the accuracy and to the list of important features.

STAGE B — From brightness to numbers

Notebook Part 2. This is the longest stage. You download about a hundred light curves, clean them, and turn each one into ten numbers that you measured yourself.

- Choose 50 confirmed planets and 50 false positives at random, using a fixed random seed so that you can repeat exactly the same selection later.
- Download each light curve with `author="Kepler"` so that every object comes from the same data pipeline.
- Remove spikes that go upwards only. The usual symmetric setting would also delete deep eclipses, which are the signals you most want to keep.
- Divide out the star's slow brightness changes with `flatten(mask=...)`, giving it a mask so that the in-transit points are ignored while the trend is being fitted.
- Write the ten feature functions listed overleaf. Three of them are left for you to complete in the notebook.
- Test your functions on a light curve you built yourself, with a transit of known depth and shape, and check that the numbers come back correct.
- Plot at least four folded light curves with the three measurement windows drawn on top, so you can see what each number is measuring.
- Run the measurement loop over the whole sample. It saves its progress as it goes, so you can stop and restart it.

Before you start Stage B, check how much data you can download

One hundred Kepler targets with eight quarters each comes to a few gigabytes and takes roughly 20 to 40 minutes on a home connection. If your connection is slow or limited, begin with `max_quarters=4` and 30 objects in each class. Get the whole pipeline working from start to finish on that small sample, then increase it. It is better to finish with a small sample than to run out of time waiting for downloads.

WEEK 2

What you do in the second week

Stage C trains the model. Stage D is where the scientific result comes from.

STAGE C — Training the model

Notebook Part 3. From here on, every number going into the model is one you measured from the brightness data. No catalogue columns are used.

- Split the data into training and test sets grouped by `kepid`, so that no star can appear on both sides of the split.
- Train the Random Forest and read the classification report. Look first at the recall for the false-positive class.
- Draw the precision-recall curve and mark the guessing baseline on it. Use this rather than a ROC curve, because the two classes are of unequal size.
- Repeat the measurement using grouped 5-fold cross-validation, which gives a fairer number because every object is tested exactly once.
- List the feature importances, then plot the two strongest features against each other and colour the two classes differently.

STAGE D — Finding the limit

Notebook Part 4. Your model now gets some objects wrong. In this stage you show that one part of that error could not have been avoided with the data you have.

- Attach the four `koi_fpflag_*` columns to your test set. These are for analysis only and must never be used for training.
- Separate the true false positives into the ones your model caught and the ones it missed.
- For each group, work out what fraction carries each of the four flags.
- Check whether `koi_fpflag_co`, the centroid-offset flag, is much more common among the ones you missed.
- Write your result as a number with an uncertainty, compared against the guessing baseline, with a physical reason for the errors that remain.

HOW TO WRITE YOUR CONCLUSION

> Using ten features measured directly from Kepler light curves, a Random Forest identified **XX% ± Y%** of false positives at a precision of **ZZ%** (grouped 5-fold cross-validation, N = ___ objects). Of the false positives the model missed, **AA%** carried the centroid-offset flag, compared with **BB%** of those it caught. A centroid offset is measured from the camera pixels rather than from the light curve, so this type of false positive cannot be identified from brightness measurements alone.

Give a number, an uncertainty, a comparison with the guessing baseline, and a reason for the errors that are left. Then say clearly what would be needed to improve the result.

CHECKING YOUR WORK

Four common mistakes

Each of these makes the results look better than they really are, which is why they are easy to miss.

Mistake 1 — using a column that contains the answer

`koi_score`, the four `koi_fpflag_*` columns and `kepler_name` were all filled in after somebody had already decided whether the object was a planet. Only confirmed planets are ever given a name, so an empty name tells you the answer directly. Including any one of these will push your score close to 100% without the model learning anything about transits.

Mistake 2 — splitting the data by row instead of by star

Some stars have more than one KOI. If `K00001.01` is used for training and `K00001.02` for testing, the model is being tested on something very similar to what it has already seen, so the score is too high. Group the split on `kepid`.

Mistake 3 — judging the model by its accuracy

About half of all KOIs are false positives, so a model that labels everything a false positive is already about 50% accurate while being useless. Report precision and recall for the false-positive class, and draw the precision-recall curve with the guessing baseline marked on it.

Mistake 4 — removing the signal along with the noise

A detrending filter that is too strong, or a symmetric outlier cut, will remove part of a deep eclipse. Always mask the transit before flattening. To check the pipeline is safe, add an artificial transit of known depth to a light curve, run it through, and confirm the depth you measure at the end is the depth you put in.

IS MY ANSWER SENSIBLE?

WHAT YOU SEE	WHAT IT PROBABLY MEANS
Recall above about 0.95	Look for a mistake before celebrating. Check that no catalogue column, flag or name has got into the feature list, and that the split is grouped on `kepid`.
Recall below about 0.5	This is usually a measurement problem rather than a model problem. Plot the folded light curves of a few objects the model got wrong and check that the features were measured on a real transit.
One feature is far more important than all the others	Either you have found something real, since depth and secondary depth do carry a lot of the signal, or that feature contains the answer. Plot it against the label and decide which.
The model gives the same answer for every object	Your sample probably became very unbalanced after removing rows with missing values. Check how many rows survived `dropna`.
Many features come out as NaN	Too few data points fell inside one of the windows. This usually means a long-period object with only two or three transits, or a wrong epoch. Check that `koi_time0bk` and the light curve use the same time system, which is BKJD for both.

GOING FURTHER

If you finish early

Four ways to extend the project, in rough order of difficulty.

Add artificial transits

Insert transits of known depth into real light curves that have already been detrended. The labels are then completely reliable and you can make as many as you like, which is what a neural network would need. Measure the depth at which your model stops finding them.

Use the camera pixels

Download the target pixel files and calculate your own centroid shift. Adding that single feature should let you catch some of the false positives you could not catch before.

Change the model

Run a Support Vector Machine and k-Nearest Neighbours on the same table of features. If the Random Forest is only slightly better, that tells you the features matter more than the choice of algorithm.

Work without labels

Run an Isolation Forest on the feature table and look at the twenty most unusual objects. Decide for each one whether it is a planet, a false positive, or something else.

FILES `KOI_false_positives_STUDENT.ipynb` is the notebook you work in. It contains six sections marked TODO for you to complete.

You will need Python 3.9 or later with lightkurve, scikit-learn, pandas and matplotlib. It runs on a laptop or in Google Colab.